

第二单元：运算符与条件判断

1. 认识运算符
2. 算术运算符
3. 赋值运算符
4. 比较运算符
 - 4.1. 隐式转换规则
5. 逻辑运算符
6. 三元（三目）运算符
7. if 语句
 - 7.1. 示例
 - 7.2. 注意事项
8. switch 语句
 - 8.1. 用法说明
9. 页面弹出框
 - 9.1. prompt 弹出输入框
 - 9.2. confirm 弹出确认框
10. 总结

1. 认识运算符

1. 什么是运算符
 - 运算符在JavaScript中用于执行算术操作、比较操作、逻辑操作等。它们是进行数据操作和实现程序逻辑的基本工具。
2. JavaScript 有哪些运算符
 - 算术运算符
 - 赋值运算符
 - 比较运算符
 - 逻辑运算符
 - 条件（三元）运算符

2. 算术运算符

- 加法 (+) : 两个数相加或字符串拼接。
- 减法 (-) : 两个数相减。
- 乘法 (*) : 两个数相乘。
- 除法 (/) : 两个数相除。
- 求模/取余 (%) : 返回两个数相除的余数。

▼ 算数运算符

JavaScript |

```
1 console.log(1 + 1) // 2
2 console.log('1' + '1') // '11'
3 console.log('a' + 'b') // 'ab'
4 console.log(1 - 1) // 0
5 console.log(2 * 3) // 6
6 console.log(6 / 3) // 2
7 console.log(10 % 3) // 1
```

- 自增 (++) : 数值增加1。

▼ 自增示例1

JavaScript |

```
1 var a = 1
2 a++ // 等同于 a = a + 1
3 console.log(2)
```

▼ 自增示例2: 后++

JavaScript |

```
1 var a = 1
2 var b = a++
3 console.log(a) // 2
4 console.log(b) // 1
5 /*
6     var b = a++
7     等同于
8     1. var b = a
9     2. a = a + 1
10    先进行赋值操作, 在进行自增
11 */
```

```
1  var a = 1
2  var b = ++a
3  console.log(a) // 2
4  console.log(b) // 2
5  /*
6      var b = ++a
7      等同于
8      1. a = a + 1
9      2. var b = a
10     先进行自增, 后进行赋值操作
11  */
```

- 自减 (--) : 数值减少1。特点与自增相同
- 注意: -, *, /, %, ++, -- 都会对操作数隐式转换为 number 在进行操作

3. 赋值运算符

- 基本赋值 (=) : 将右侧的值赋给左侧的变量。
- 加法赋值 (+=) : 加上右侧的值后再赋值。
- 减法赋值 (-=) : 减去右侧的值后再赋值。
- 乘法赋值 (*=) : 乘以右侧的值后再赋值。
- 除法赋值 (/=) : 除以右侧的值后再赋值。

```
1  var a = 1 // 把 1 赋值给变量 a
2  a += 2 // 等同于 a = a + 2
3  a -= 2 // 等同于 a = a - 2
4  a *= 2 // 等同于 a = a * 2
5  a /= 2 // 等同于 a = a / 2
```

4. 比较运算符

- 等于 (==) : 检查两个值是否相等, 类型不同会自动转换类型在比较, 大部分是转换成 number。
- 全等 (===) : 检查值和类型是否完全相同, 类型不同直接返回false, 不会转换类型。
- 不等于 (!=) : 检查两个值是否不相等, == 的反操作

- 不全等 (!==) : 检查值或类型是否不完全相同, === 的反操作
- 大于 (>)、小于 (<) : 数值大小比较。
- 大于等于 (>=)、小于等于 (<=) : 数值大小或等值比较。
- 比较运算符只能返回布尔值, true / false

▼ 比较运算符 JavaScript

```
1 console.log(1 == '1') // true, 转成number比较
2 console.log(1 === '1') // false, 类型不同
3 console.log(1 != '1') // false, 转成number比较
4 console.log(1 !== '1') // true, 类型不同
5 console.log(2 > 1) // true
6 console.log(2 < 1) // false
7 console.log(2 >= 1) // true
8 console.log(2 <= 1) // false
9
```

4.1. 隐式转换规则

- 使用 ==、!=、>、>=、<、<= 比较时如果两边的数据类型不同, JavaScript会进行隐式类型转换成相同类型然后进行比较

1. 字符串与数字: 字符串会被转换为数字。

- 例: '5' == 5 会变为 5 == 5。

2. 布尔值与其他类型: 布尔值会被转换为数字, 然后进行比较 (true转换为1, false转换为0)。

- 例: true == 1 会变为 1 == 1。
- 例: false == 0 会变为 0 == 0。

3. 对象与原始类型: 对象会通过调用其toString()或valueOf()方法转换为原始类型 (字符串或数字), 然后进行比较。

- 例: [1] == 1 会首先将数组[1]转换为字符串 "1", 然后将字符串转换为数字1。

4. null与undefined: null和undefined在使用==比较时互相相等, 但不等于其他任何类型。

- 例: null == undefined 结果为 true。
- 但 null 或 undefined 与其他类型比较时, 结果总是 false。

5. NaN: NaN与任何值 (包括其自身) 比较的结果都是false。

- 例: NaN == NaN 结果为 false。

这些隐式转换规则可能会导致一些不直观的比较结果，在实际编程中推荐使用**全等运算符 === 比较**，使用 `===` 可以避免隐式转换带来的意外和错误。

5. 逻辑运算符

- 与 (`&&`) :

- 所有条件都为真时返回最后一个真值，否则返回找到的假值。
- 如果第一个操作数为假值，则不会检查第二个操作数。

▼ 逻辑与

JavaScript |

```
1 console.log(true && true);    // 输出: true
2 console.log(true && false);   // 输出: false
3 console.log(false && true);   // 输出: false
4 console.log(false && false);  // 输出: false
5 console.log(0 && 100);        // 输出: 0
6 console.log(1 && 100);        // 输出: 100
```

- 或 (`||`) :

- 条件中任意一个值为真时返回该真值，没有真值返回最后一个假值。
- 如果第一个操作数为`true`，则不会检查第二个操作数。

▼ 逻辑或

JavaScript |

```
1 console.log(true || true);   // 输出: true
2 console.log(true || false);  // 输出: true
3 console.log(false || true);  // 输出: true
4 console.log(false || false); // 输出: false
5 console.log(0 || 100);        // 输出: 100
6 console.log(1 || 100);        // 输出: 1
```

- 非 (`!`) :

- 逻辑非 用于反转操作数的布尔值。
- 如果操作数为真，则返回`false`；如果操作数为假，则返回`true`。
- 先将值隐式转换为布尔值再取反

```
1 console.log(!true); // 输出: false
2 console.log(!false); // 输出: true
3 console.log(!0); // 输出: true
4 console.log(!null); // 输出: true
5 console.log(!undefined); // 输出: true
6 console.log(!!0); // 输出: false
```

6. 三元（三目）运算符

- 基本语法是：条件 ? 表达式1 : 表达式2。
- 如果条件为真，则执行并返回表达式1的结果；如果条件为假，则执行并返回表达式2的结果。

```
1 var a = 10, b = 5;
2 var result = a > b ? "a的值比较大" : "b更大一点";
3 console.log(result); // 输出: "a的值比较大"
```

7. if 语句

- 基本语法

```
1 if (条件1) {
2     // 当条件1为真时执行的代码
3 } else if (条件二) {
4     // 当条件1为假，但条件2为真时执行的代码
5 } else {
6     // 当所有条件都为假时执行的代码
7 }
```

7.1. 示例

1. 基本的if语句：

▼ 单分支

JavaScript |

```
1 let score = 75;
2 if (score > 70) {
3     console.log("Pass");
4 }
5 // 如果score大于70, 则输出"Pass"。
```

2. 带else的if语句:

▼ 双分支

JavaScript |

```
1 let age = 18;
2 if (age >= 18) {
3     console.log("Adult");
4 } else {
5     console.log("Minor");
6 }
7 // 如果age大于或等于18, 输出"Adult"; 否则输出"Minor"。
```

3. 带else if和else的完整if语句:

▼ 多分支

JavaScript |

```
1 let number = 15;
2 if (number > 20) {
3     console.log("大于20");
4 } else if (number > 10) {
5     console.log("大于10并且小于等于20");
6 } else {
7     console.log("10以下");
8 }
9 // 根据number的值, 输出相应的信息。
```

7.2. 注意事项

- if语句中的条件应该是一个返回布尔值的表达式。
- 在JavaScript中, 只有当条件为真 (或真值, 即非false, 0, "", null, undefined, NaN) 时, if语句中的代码才会被执行。

8. switch 语句

switch 语句用于基于不同的条件执行不同的操作，其基本语法如下：

▼ switch

JavaScript |

```
1 switch (表达式) {  
2     case 值1:  
3         // 表达式的值 === 值1 时执行此处代码  
4         console.log(1)  
5         break;  
6     case 值2:  
7         // 表达式的值 === 值2 时执行此处代码  
8         console.log(3)  
9         break;  
10    // 可以有任意数量的case语句  
11    default:  
12        // 如果表达式的值与所有case都不匹配时执行的代码  
13 }  
14
```

8.1. 用法说明

- 示例中表达式值将与每个**case**后面的值进行比较，使用的是 **全等(===)** 比较
- **case**：代表一个与**表达式**比较的值。如果匹配，执行该**case**后的代码。
- **break**：用于退出**switch**语句。如果没有**break**，程序将继续执行下一个**case**的代码，不论其值是否匹配。
- **default**（可选）：当没有任何**case**匹配时，执行的代码块。

9. 页面弹出框

9.1. prompt 弹出输入框

- `prompt('提示信息')` 用于显示一个对话框，可让用户输入文本。
- 点击确定把用户输入的内容以字符串形式返回，点击取消返回 **null**

▼ prompt

JavaScript |

```
1 let age = prompt("请输入你的年龄");  
2 console.log(age); // 打印用户输入的年龄
```


9.2. confirm 弹出确认框

- `confirm('提示信息')`
- 点击确定返回 `true`，点击取消返回 `false`

▼ confirm

JavaScript |

```
1 let isAdult = confirm("你确定是你年满18岁了吗?");
2 console.log(isAdult); // 如果用户确认，则打印 true，否则打印 false
```

10. 总结

1. 5种运算符的使用场景和特点。
2. 运算符隐式转换规则。
3. if 语句条件控制语句。
4. switch 的用法和特点。
5. prompt 和 confirm 弹出框。